

SAVVYDFIR-MCP-Analysis

SAVVYDFIR-MCP: Kismat's Plan -- In-Depth Analysis & Architectural Critique

[Back to Index](#)

[FindEvil-Hackathon](#) | [Architecture-Brainstorm](#)

Purpose

Line-by-line analysis of Kismat's SAVVYDFIR-MCP architecture, what it does well, and where it contradicts the Claude Architect Playbook. Written from Yash's perspective: prompt-level guardrails are naïve -- structural enforcement is the standard.

Part 1: What Kismat Built

Source: `/Documents/projects/FIND-EVIL/KISMAT/SAVVYDFIR_MCP_Overview.md`

Repo: github.com/kismatkunwar89/SAVVYDFIR-MCP

SAVVYDFIR-MCP is a 31-tool forensic MCP server that turns Claude Code into an autonomous DFIR analyst. You provide evidence (disk images, memory dumps, EVTX logs, PCAPs), write a manifest, run `claude`, and it autonomously triages, analyzes, correlates, and reports.

Part 2: Component-by-Component Analysis

2.1 MCP Server (`server.py`) -- 31 Forensic Tools

What it is: A Python MCP server that exposes 31 functions over `stdio`. Each function wraps a forensic binary (`MFTECmd`, `vol`, `chainsaw`, `tshark`, etc.) via `subprocess.run()`, parses the output, wraps it in a Forensic Envelope, and returns structured JSON.

What it does well:

- **Tool coverage is excellent.** 12 disk, 5 memory, 3 network, 3 threat detection, 5 reporting, 3 meta. This covers the full DFIR workflow -- from initial triage to final report. No missing artifact types.
- **Parsing logic is the hard part, and Kismat did it.** Turning raw CSV/text output from EZ tools and Volatility into structured JSON is tedious, error-prone work. This is genuinely valuable.
- **Case-agnostic design.** No hardcoded hostnames, IPs, or paths. Everything derives from the manifest and `SAVVYDFIR_ANALYSIS_DIR`. Clean separation of config from code.

Concerns:

- **All 31 tools are exposed to a single orchestrator.** Claude Code sees all 31 tool schemas in its system prompt (~5K-6K tokens of schema overhead per turn, whether it needs the tools or not).
- **No tool scoping by investigation phase.** When analyzing disk artifacts, Claude still sees all 5 memory tools and 3 network tools. This increases selection confusion and wastes context.
- **Subprocess calls are black boxes.** If `MFTECmd` fails, the error handling is whatever `server.py` implements. No retry logic, no fallback, no partial result handling is described.

2.2 Forensic Knowledge System (Forensic Envelope)

What it is: Every tool response includes four fields injected from a Valhuntr MIT YAML knowledge base + SANS FOR508 content:

Field	Purpose
forensic_caveat	Artifact-specific limitations and gotchas
corroborate_with	Other artifacts to cross-reference
discipline_reminder	Anti-pattern warnings from SANS methodology
data_provenance	Tool version, command line, evidence path, timestamp

What it does well:

- **Forensic discipline is real.** DFIR has actual methodology requirements -- chain of custody, corroboration, caveat documentation. The envelope encodes these at the protocol level. Every tool response tells Claude what it CAN'T conclude from this artifact alone.
- **Corroboration pointers are valuable.** corroborate_with: ["prefetch", "amcache", "evtx"] tells Claude exactly which tools to call next. This is effectively a domain-specific tool-chaining graph encoded in YAML.
- **Data provenance is competition-ready.** Judges can see exactly which binary, which version, which command line produced each finding. Chain-of-custody documentation baked in.

Concerns:

- **Corroboration is advisory, not enforced.** The envelope says "corroborate with prefetch" but nothing PREVENTS Claude from marking a finding as high-confidence without doing so. It's a prompt hint. Claude can (and will, under context pressure) skip corroboration and still produce findings.
- **Caveat fatigue.** After 15+ tool calls, each with a caveat and discipline reminder, Claude starts treating these as boilerplate. The model learns to skim them the same way humans learn to skim cookie banners. The 12th "single-artifact evidence is never conclusive" reminder has less effect than the 1st.
- **~200 tokens per envelope x 30+ calls = 6K+ tokens of envelope text in context that serves a prompt-level function.** In a structural approach, caveats are encoded as constraints in the data schema, not as natural language warnings.

2.3 Claude as Orchestrator (CLAUDE.md)

What it is: Claude Code reads CLAUDE.md at session start. This file contains the investigation protocol: how to interpret the manifest, evidence triage order, findings management, and when to escalate to specialist agents. Claude IS the orchestrator -- no custom code controls the investigation flow.

What it does well:

- **Zero orchestration code.** You don't write a runner, scheduler, or state machine. Claude reads the protocol and figures out the workflow. This is fast to build and easy to change (edit CLAUDE.md, done).
- **Natural language flexibility.** If the investigation goes sideways, Claude can adapt -- it's not locked into a fixed pipeline. It can re-prioritize, skip irrelevant tools, or deep-dive on unexpected findings.

Concerns:

- **Claude Code IS the bottleneck.** Every decision (what tool to call, how to interpret results, what to investigate next) runs through a single Claude context window. There's no parallelism, no concurrent investigation tracks. Tool calls are sequential.
- **No separation of reasoning from execution.** Claude simultaneously decides what to do, calls the tool, interprets the result, and plans the next step. In the playbook, these are separate agents with different models and toolsets. Mixing them in one context means reasoning competes with tool output for context space.
- **Protocol compliance is hope-based.** CLAUDE.md says "follow triage order, write findings to state.json, corroborate before high-confidence." But nothing enforces this. Claude might skip steps, forget to write findings, or mark things high-confidence without corroboration. The protocol is a suggestion, not a constraint.

2.4 PostToolUse Specialist Agents (11 Agents)

What it is: After each tool call completes, a PostToolUse hook spawns a specialist sub-agent that receives the raw tool output and runs focused analysis. Agents like `evtx-analyst`, `mft-analyst`, `memory-analyst` are domain-specific.

Agent	Triggered by	Focus
<code>evtx-analyst</code>	<code>summarize_evtx</code>	Auth anomalies, lateral movement, persistence
<code>mft-analyst</code>	<code>extract_mft_timeline</code>	Timeline gaps, suspicious file ops, deletion
<code>registry-analyst</code>	<code>extract_registry</code>	Persistence keys, autoruns, shellbags
<code>prefetch-analyst</code>	<code>extract_prefetch</code>	Execution timeline, LOLBin detection
<code>memory-analyst</code>	<code>analyze_memory</code>	Injection, hollowing, suspicious connections
<code>amcache-analyst</code>	<code>get_amcache</code>	Execution evidence, hash correlation
<code>srum-analyst</code>	<code>extract_srum</code>	Long-term behavioral baseline
<code>browser-analyst</code>	<code>extract_browser_artifacts</code>	Phishing, downloads, credential exposure
<code>timeline-analyst</code>	Multiple	Cross-artifact super-timeline
<code>corroboration-analyst</code>	Any finding	Cross-artifact validation
<code>sigma-analyst</code>	<code>sigma_hunt</code>	Chainsaw hit triage, false positive reduction

What it does well:

- **Domain specialization is correct.** An EVTX-focused agent with EVTX-specific instructions will produce better analysis than a generalist agent looking at everything. This follows the multi-agent specialization principle.
- **Automatic triggering removes human-in-the-loop.** You don't have to remember to run deep analysis on EVTX results -- the hook fires automatically. This is good UX for autonomous operation.
- **Corroboration agent is the right idea.** Having a dedicated agent whose job is cross-artifact validation is architecturally sound.

Concerns:

- **Agents are reactive, not proactive.** They fire AFTER Claude already called the tool. The orchestrator (Claude) decided to run `summarize_evtx` -- the `evtx-analyst` agent doesn't get to decide which Event IDs to focus on or what time window matters. It gets the full dump and has to make sense of it.
- **Each agent gets the FULL raw tool output.** If `extract_mft_timeline` returns 100K tokens of MFT data, the `mft-analyst` agent receives all 100K tokens. No pre-filtering, no summary. The agent pays the full token cost for data that's 95% noise.
- **Agents can't talk to each other.** The `evtx-analyst` finds a suspicious logon at 14:30. The `mft-analyst` finds a suspicious file creation at 14:32. These two agents don't share findings directly -- they both write to `state.json`, and it's up to the orchestrator (Claude) or the `corroboration-analyst` to connect them. There's no structured inter-agent communication channel.
- **Duplicate reasoning.** Claude (orchestrator) interprets the EVTX results. Then `evtx-analyst` ALSO interprets the same EVTX results. Two models reasoning over the same data -- double the token cost for potentially the same conclusions.

2.5 Investigation State (`state.json` + `audit.jsonl`)

What it is: Two files persist state across the investigation:

- `state.json`: Live findings database. Each finding has: tool, confidence, MITRE tactic, supporting artifacts, analyst notes, and graph data.
- `audit.jsonl`: Append-only log of every tool call with arguments, response hash, and timestamp.

What it does well:

- **Structured findings are correct.** Each finding carries typed metadata (confidence, MITRE mapping, supporting artifacts). This is better than free-text conclusions.
- **Audit trail is competition-critical.** Judges need to see HOW the agent reached its conclusions. `audit.jsonl` with full tool call records provides this.
- **Graph data in state.json** feeds the D3.js visualization. Investigation graphs are a strong differentiator for the competition.

Concerns:

- **state.json is written by Claude.** The `add_finding` tool is called by Claude with parameters Claude decides. If Claude hallucinates a finding, it gets written to state.json with whatever confidence level Claude assigns. The state file has no validation logic -- it trusts whatever Claude sends.
 - **No finding deduplication.** If Claude calls `extract_registry` twice (e.g., after a re-triage), it might add duplicate findings. No structural dedup.
 - **Confidence is a string, not computed.** A finding is "high", "medium", or "low" confidence based on Claude's judgment. In a structural approach, confidence would be computed: high = corroborated by 2+ artifact types (code checks this), medium = single strong indicator, low = anomaly only. Here, Claude just picks a word.
-

2.6 Multi-Host Architecture

What it is: Each host runs in its own Claude Code session with its own `SAVVYDFIR_ANALYSIS_DIR`. After all hosts complete, `merge_host_graphs()` and `build_reports_index()` unify the results.

What it does well:

- **Isolation is correct.** Each host analysis is independent, can run in parallel (separate terminals), and has its own state.
- **Unified dashboard is competition-ready.** Cross-host lateral movement visualization in `D3.js` is a strong deliverable.

Concerns:

- **No cross-host correlation during analysis.** Host A's session doesn't know what Host B found. Lateral movement detection requires knowing that User X logged into Host B at 14:30 (from Host B's EVTX) AND that Host A initiated an SMB connection to Host B at 14:28 (from Host A's network artifacts). These two findings exist in separate Claude sessions. The merge happens AFTER analysis, not during.
 - **Sequential host analysis loses context.** If Host A analysis reveals IOCs, those should seed Host B's investigation. The manifest supports `known_iocs`, but there's no automated pipeline to extract IOCs from Host A's state.json and inject them into Host B's manifest.
-

2.7 Reporting Pipeline

What it is: Five tools handle output generation:

- `generate_graph`: Per-host `D3.js` investigation graph
- `merge_host_graphs`: Cross-host lateral movement graph
- `build_reports_index`: Unified HTML dashboard
- `synthesize_findings`: Structured DFIR report from state.json
- `add_finding` / `update_finding`: Finding CRUD

What it does well:

- **D3.js graphs are visually compelling.** This is a hackathon -- judges see the demo. Interactive investigation graphs differentiate from text-only reports.
- **Structured report from structured findings is the right pipeline.** Findings -> synthesis -> report, not free-text reasoning -> report.

Concerns:

- **Report quality depends on finding quality.** If findings are hallucinated or low-quality (see Section 2.5), the report inherits those problems with a professional-looking wrapper. This can be worse than no report -- it creates false confidence.
-

Part 3: Architectural Critique -- Where It Contradicts the Claude Architect Playbook

Contradiction 1: 31 Tools in One Context

Playbook says: "Give agents focused toolsets. Narrow tool scope per agent = better tool selection, fewer hallucinated calls, less prompt overhead."

SAVVYDFIR does: Exposes all 31 tools to a single Claude Code session. Claude sees disk, memory, network, threat detection, and reporting tools simultaneously, regardless of investigation phase.

Why this matters: Claude's tool selection accuracy degrades with tool count. At 5-8 tools, selection is reliable. At 31, the model burns reasoning tokens deciding between `extract_shimcache` vs `get_amcache` vs `extract_prefetch` -- all of which surface execution evidence with subtle differences. The playbook explicitly warns against this pattern.

Structural fix: Split tools across domain-specific agents. Disk agent sees 12 tools. Memory agent sees 5. No agent sees all 31.

Contradiction 2: Prompt-Level Guardrails Instead of Structural Enforcement

Playbook says: "Don't rely on telling the model what not to do. Encode constraints in tool schemas, output formats, and orchestration logic."

SAVVYDFIR does: Uses the Forensic Envelope (`forensic_caveat` , `discipline_reminder`) and CLAUDE.md protocol instructions to prevent hallucination and enforce forensic discipline.

Why this is naive (Yash's thesis): Telling Claude "single-artifact evidence is never conclusive" in every tool response is the same pattern as writing "DON'T HALLUCINATE" in a system prompt. It works most of the time. It fails exactly when it matters most -- under context pressure, in long investigations, when the model is trying to produce a coherent narrative from ambiguous evidence.

The Forensic Envelope is a more sophisticated version of prompt-level guardrails (domain-specific, per-tool, with corroboration pointers), but it's still fundamentally: the model **CHOOSSES** whether to follow the guidance. There's no code that prevents Claude from writing a high-confidence finding without corroboration.

Structural alternative (Wraith pattern):

```
# Code-enforced corroboration
class Finding:
    confidence: Literal["high", "medium", "low"]
    supporting_artifacts: List[ToolFinding]

    @validator("confidence")
    def enforce_corroboration(cls, v, values):
        artifacts = values.get("supporting_artifacts", [])
        unique_tools = set(f.tool for f in artifacts)
        if v == "high" and len(unique_tools) < 2:
            raise ValueError("High confidence requires 2+ artifact types")
        return v
```

The model literally cannot output a high-confidence finding without 2+ supporting artifact types. Not because the prompt says so -- because the schema validator rejects it.

Contradiction 3: Single-Agent Orchestration

Playbook says: "Use multi-agent architectures for complex workflows. Separate planning from execution. Use different models for different cognitive tasks."

SAVVYDFIR does: Claude Code (single model, single context) is simultaneously the planner, executor, interpreter, and reporter. CLAUDE.md is the "protocol," but it's all one brain.

Why this matters:

- **No model tiering.** Tool calls don't need Opus-level reasoning -- Haiku can call `extract_mft_timeline` just fine. Interpretation and hypothesis generation DO need Opus. Mixing them in one session means you pay Opus prices for tool calls.
- **Context competition.** Tool results compete with reasoning for context space. After 10 tool calls, Claude's context is dominated by raw forensic data. Its reasoning quality degrades because the important analysis is buried under MFT timestamps.
- **No parallelism.** Disk analysis and memory analysis are independent. They could run concurrently. Single-agent orchestration makes them sequential.

Structural fix: The architecture from [Architecture-Brainstorm](#):

```
Orchestrator (code, not Claude) -----> hypothesis_agent (Opus, no tools)
                                         |---> disk_agent (Sonnet, 12 tools)
                                         |---> memory_agent (Sonnet, 5 tools)
                                         |---> network_agent (Haiku, 3 tools)
                                         |---> threat_agent (Sonnet, 3 tools)
                                         |---> report_agent (Sonnet, 5 tools)
```

Contradiction 4: No Output Schema Enforcement

Playbook says: "Use constrained decoding / output schemas to ensure model outputs are structurally valid."

SAVVYDFIR does: Claude writes findings to `state.json` via the `add_finding` MCP tool. The tool accepts whatever Claude sends -- title, description, confidence, MITRE mapping -- as free-form parameters. No schema validation on the model's output.

Why this matters: Claude can:

- Write a finding with `confidence: "high"` without any corroboration
- Map to the wrong MITRE technique (T1059 vs T1059.001)
- Write a description that doesn't match the supporting artifacts
- Add duplicate findings with slightly different wording

Structural fix (Wraith v10.0d pattern): Constrained decoding via Agent SDK `output_format`:

```
output_format = {
  "type": "object",
  "properties": {
    "title": {"type": "string", "maxLength": 100},
    "confidence": {"enum": ["high", "medium", "low"]},
    "mitre_tactic": {"pattern": "^TA[0-9]{4}$"},
    "mitre_technique": {"pattern": "^T[0-9]{4}(\\. [0-9]{3})?$"},
    "supporting_artifacts": {
      "type": "array",
      "minItems": 1,
      "items": {"type": "string"}
    }
  },
  "required": ["title", "confidence", "mitre_tactic", "supporting_artifacts"]
}
```

The model **MUST** produce valid JSON matching this schema. Invalid MITRE IDs, missing artifacts, or malformed confidence levels are rejected before they hit `state.json`.

Contradiction 5: No Token Efficiency Strategy

Playbook says: "Be deliberate about what enters the model's context. Filter tool outputs. Summarize before feeding to the next agent."

SAVVYDFIR does: Full tool output enters Claude's context. Full tool output is passed to specialist agents. No pre-filtering, no summarization layer, no output truncation.

Why this matters:

- `extract_mft_timeline` on a real disk can produce 500K+ MFT entries. Even filtered to JSON, this is 50K-100K+ tokens.
- This data sits in context for EVERY subsequent turn (cached, but still paid for as input tokens).
- Specialist agents receive the same raw data -- no summary or filtering between orchestrator and agent.
- 31 tool schemas (~5K tokens) baked into every API call.

Conservative token estimate for one investigation:

```
Tool schemas:                ~5K tokens (every turn)
15 tool calls x ~20K avg result: ~300K tokens
Forensic Envelopes:         ~6K tokens
11 specialist agents:        each gets full tool output
Claude reasoning (15 turns): ~30K tokens output

Total: 500K - 1M+ input tokens per single-host investigation
At Opus: $7-15 per case
At Sonnet: $1.50-3 per case
```

Multiply by host count for multi-host scenarios. For a hackathon demo with 3-5 hosts, that's \$20-75 per run at Opus.

Structural fix: Preprocessing layer (Wraith v10.0b pattern):

```
# Tool output goes to file (0 tokens)
raw_output = subprocess.run(["MFTCmd", ...], capture_output=True)
write_to_file(raw_output, "mft_raw.csv")

# Preprocessor filters to relevant findings (500 tokens instead of 50K)
findings = preprocessor.parse("mft_raw.csv", filters={
    "time_window": incident_range,
    "suspicious_only": True
})

# Agent sees summary, not raw data
agent_context = findings.to_summary() # ~500 tokens
```

Part 4: What to Take, What to Replace

Keep (Kismet's Strengths)

1. **31 tool implementations** -- the parsing logic, binary invocations, output structuring
2. **Forensic Envelope concept** -- but encode as schema constraints, not prompt text
3. **Case-agnostic design** -- manifest-driven, no hardcoded values
4. **D3.js investigation graphs** -- competition differentiator
5. **Audit trail (audit.jsonl)** -- chain-of-custody documentation
6. **Corroboration pointers** -- the knowledge of which artifacts validate which

Replace (With Playbook-Compliant Architecture)

1. **Claude Code as orchestrator** -> Agent SDK orchestrator (code controls flow)
2. **31 tools in one context** -> Domain-scoped agents (3-12 tools each)
3. **Prompt-level guardrails** -> Schema-enforced constraints (constrained decoding)
4. **Full output in context** -> Preprocessing + filtered summaries
5. **Sequential single-host** -> Parallel host analysis with cross-host IOC sharing
6. **Reactive specialist agents** -> Proactive domain agents that decide their own tool calls
7. **String confidence levels** -> Computed confidence from artifact corroboration count

Summary

Kismat built excellent forensic **tooling**. The 31 tools, parsing logic, envelope knowledge base, and visualization pipeline are the hard part -- and they're done. The architectural weakness is in **orchestration**: using Claude Code as a single-agent orchestrator with prompt-level guardrails contradicts every principle in the Claude Architect Playbook.

The playbook-compliant architecture: keep Kismat's tools as the execution layer, wrap them in an Agent SDK orchestrator with structural guardrails, domain-scoped agents, constrained decoding, and a preprocessing layer that controls what enters context.

Yash's thesis is correct: telling a model "don't hallucinate" in the system prompt (even with sophisticated domain-specific framing like the Forensic Envelope) is fundamentally naive. The model chooses whether to comply. Structural enforcement -- schema validation, constrained decoding, code-level corroboration checks -- removes the choice. The model CAN'T hallucinate because the architecture won't let it.

Related

- [Architecture-Brainstorm](#) -- Our Agent SDK + MCP two-layer architecture
- [FindEvil-Hackathon](#) -- Hackathon hub
- [Literature-Review](#) -- 25-paper evidence base for multi-agent forensic AI